TECHLEARN ACADEMY

Árvores binárias e seus percursos

Curso

Lógica de Programação e Estruturas de Dados

Módulo 4 - Filas e árvores binárias

Como estudar esta matéria

1. Leia a explicação teórica e acompanhe os exemplos do material.

2. Resolva os exercícios comentados antes de conferir suas justificativas. Anote suas dúvidas para conversar com o mentor.

3. Ao terminar os materiais do módulo, faça a avaliação de revisão disponível no site.

Origem do conteúdo

CQA/UNIP - Tecnologia em Análise e Desenvolvimento de Sistemas, Tomo 10. Páginas 20 a 23 do arquivo original.

O conteúdo da matéria começa na página 4 deste PDF. Antes dele, foram mantidas a capa e a página de créditos e avisos do documento original.

Os textos e exercícios originais conservam suas referências e o contexto de publicação. A organização em cursos e as avaliações de revisão pertencem ao projeto TechLearn Academy.

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTRUCIONAL ESPECÍFICO

TOMO 10

CQA/UNIP – Comissão de Qualificação e Avaliação da UNIP

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTRUCIONAL ESPECÍFICO

TOMO 10

Christiane Mazur Doi

Doutora em Engenharia Metalúrgica e de Materiais, Mestra em Ciências - Tecnologia Nuclear, Especialista em Língua Portuguesa e Literatura, Engenheira Química e Licenciada em Matemática, com Aperfeiçoamento em Estatística. Professora titular da Universidade Paulista.

Tiago Guglielmeti Correale

Doutor em Engenharia Elétrica, Mestre em Engenharia Elétrica e Engenheiro Elétrico (ênfase em Telecomunicações). Professor titular da Universidade Paulista.

Material instrucional específico, cujo conteúdo integral ou parcial não pode ser reproduzido ou utilizado sem autorização expressa, por escrito, da CQA/UNIP – Comissão de Qualificação e Avaliação da UNIP – UNIVERSIDADE PAULISTA.

Questão 6

Questão 6.[6]

O fragmento de código C++, a seguir, contém a implementação da inserção de um nó em uma árvore binária.

```
...
struct noArvore {
      int dado;
      noArvore *esquerda;
      noArvore *direita;
};
noArvore *insere(noArvore *arvore, int valor) {
      if (arvore == NULL) {
            arvore = new noArvore;
            arvore->esquerda = NULL;
            arvore->direita = NULL;
            arvore->dado = valor;
      } else if(valor < arvore->dado) {
            arvore->esquerda = insere (arvore->esquerda, valor);
      } else {
            arvore->direita = insere (arvore->direita, valor);
      }
      return(arvore);
}
...
```

Com base no exposto, assinale a opção que apresenta o fragmento de programa para imprimir os valores contidos nos nós, percorrendo a árvore em pré-ordem.

A.
```
void preorder(noArvore *raiz) {
      if (raiz != NULL) {
            cout << raiz->dado << " ";
            preorder(raiz->esquerda);
            preorder(raiz->direita);
      }
   }
```

B.
```
void preorder(noArvore *raiz) {
   if (raiz != NULL) {
      preorder(raiz->direita);
      preorder(raiz->esquerda);
      cout << raiz->dado << " ";
   }
```

[6]Questão 25 – Enade 2017.

```
      }
C.   void preorder(noArvore *raiz) {
         if (raiz != NULL) {
            preorder(raiz->esquerda);
            cout << raiz->dado << " ";
            preorder(raiz->direita);
         }
      }
D. void preorder(noArvore *raiz) {
      if (raiz != NULL) {
            preorder(raiz->esquerda);
            preorder(raiz->direita);
            cout << raiz->dado << " ";
      }
   }
E. void preorder(noArvore *raiz) {
         if (raiz != NULL) {
            preorder(raiz->direita);
            cout << raiz->dado << " ";
            preorder(raiz->esquerda);
         }
      }
```

1. Análise da questão

Percurso de árvores binárias

Árvores binárias são estruturas de dados compostas por nós interligados de forma hierárquica. Em uma árvore que não esteja completamente vazia, um desses nós é chamado de raiz, e novos nós são adicionados como nós filhos desse nó principal. Além disso, nas árvores binárias, existe a restrição de que cada nó pode ter zero, um ou, no máximo, dois nós filhos.

O percurso de uma árvore binária envolve visitar cada um dos seus nós e processá-los uma única vez (VEERARAJAN, 2007). Diferentemente de uma estrutura de dados linear, na

qual existe uma forma óbvia de fazer um percurso, em uma árvore binária temos várias possibilidades de caminhos.

No caso de uma árvore binária, cada uma das possibilidades de processamento é uma combinação de três possibilidades:

- processamento de dado nó atual,
- visita ao nó filho esquerdo (se ele existir);
- visita ao nó filho direito (se ele existir).

A ordem na qual essas operações podem ser feitas define os tipos de percursos. Nesse sentido, temos as seguintes opções:

- percurso em pré-ordem;
- percurso em ordem;
- percurso em pós-ordem.

O percurso em pré-ordem é aquele no qual primeiro se processa o nó visitado, depois se visita o nó filho esquerdo e, posteriormente, visita-se o nó filho direito. Outra possibilidade é o percurso em-ordem, no qual primeiro se visita o nó filho esquerdo, depois se processa o nó atual e, em seguida, visita-se o nó direito. Finalmente, o percurso em pós-ordem é aquele no qual se visita o nó filho esquerdo, depois o nó filho direito e, por último, processa-se o nó atual.

2. Resolução da questão

Os fragmentos de código das alternativas apresentam possíveis soluções para o problema de percurso da árvore binária com o uso da recursividade. Devemos lembrar que o percurso em pré-ordem envolve primeiro o processamento do nó atual. No caso do exercício, isso se refere simplesmente a mostrar na tela o conteúdo do número inteiro armazenado em raiz->dado, o que pode ser feito pela linha de código indicada a seguir.

```
cout << raiz->dado << " ";
```

Depois disso, devemos visitar o nó esquerdo, chamando a mesma função (*preorder*, no código) de forma recursiva. Finalmente, devemos visitar o nó direito. Isso pode ser feito pelas linhas de código indicada a seguir.

```
preorder(raiz->esquerda);
preorder(raiz->direita);
```

Contudo, temos mais um problema a ser evitado: em linguagens como C e C++, nas quais trabalhamos com ponteiros, não podemos acessar o conteúdo de um ponteiro nulo, ou seja, de um ponteiro que aponta para NULL. Se tentarmos fazer isso, teremos um erro de execução. Assim, devemos proteger o código. Para isso, verificamos, previamente, se o ponteiro para o nó a ser processado é nulo ou não, o que pode ser feito pela indicação a seguir.

```
if( raiz != NULL )
```

Com base nisso, identificamos o código da alternativa A como correto.

Alternativa correta: A.

3. Indicações bibliográficas

- AGUILAR, L. J. *Programação em C++: algoritmos, estruturas de dados e objetos.* 2. ed. Porto Alegre: AMGH, 2011.
- CELES, W.; CERQUEIRA, R.; RANGEL, J. R. *Introdução à estrutura de dados*. Rio de Janeiro: Campus, 2004.
- TUCKER, A. B; NOONAN, R. E. *Linguagens de programação: princípios e paradigmas.* 2. ed. São Paulo: MaGrawhill, 2008.
- VEERARAJAN, T. *Discrete Mathematics with graph theory and combinatorics*. New Delhi: Tata McGraw-Hill, 2007.